How Much Is Per-Image Step Placement Worth?

Karthik Ravula · B.Tech. in Artificial Intelligence, IIT Hyderabad (2025)
interact513@gmail.com · Code on GitHub · October 2026

Question. At a fixed step budget, how much does tailoring the schedule to each image improve on a shared schedule optimized for the same budget?

Answer. At 4 and 8 steps, images often prefer different schedules, yet the average per-image gain was smaller than one extra denoising step on the shared schedule (full sampling, LSUN Church and Bedroom).

Motivation

Diffusion models generate an image through many denoising steps, and the number of steps largely sets the cost of sampling. Standard fixed-schedule samplers use the same denoising schedule for every image, regardless of its complexity. That suggests an opportunity: a sampler that allocates steps per image, spending fewer on easy images and more on hard ones, could cut compute without losing quality. Across my evaluations, this idea led to a more basic question: how much is per-image allocation actually worth?

Learning to skip steps (Computer Vision course project – TAED)

My first approach let a reinforcement-learning agent decide, at each DDPM step, whether to denoise or skip. This was a four-person Computer Vision course project at IIT Hyderabad (Prof. Sumohana Channappayya). I designed the MDP and trained the final A2C policy (CNN + LSTM) on MNIST, comparing REINFORCE, A2C and PPO. The best constrained policy cut sampling time from 13.5 s to 10.24 s, but FID rose from 5.59 to 20.49. Reward design was the main difficulty: rewarding only the final image made individual decisions hard to credit, while per-step rewards did not always reflect image quality.

Planning skips on latent diffusion (independent work)

Building on the course project, I continued on my own with latent diffusion models and DDIM sampling, and worked on three limitations of the original design:

- **Expensive image features.** The LSTM processed full-image features before every step. I replaced it with a lightweight latent encoder.
- **Weak reward design.** I examined how well each reward tracked final perceptual quality, and the trade-off between sparse terminal rewards and dense per-step feedback. I tried LPIPS-based perceptual similarity, classifier confidence, feature-manifold proximity and image-quality heuristics, combined in weighted objectives with a denoising-cost penalty.
- **A decision at every timestep.** Inspired by Diffusion Policy, the planner proposes a sequence of denoise-or-skip decisions from the current state, executes a short segment, then replans, so the policy runs less often while the actions stay binary.

With these changes, the planner skipped 60–70% of a 200-step schedule but still needed 60–80 steps, while solvers such as DPM-Solver++ report high-quality samples in 15–20 steps. Because the step count was an outcome of the policy rather than a setting, I added a compute budget to the MDP. The policy struggled to use it, and its decisions barely depended on the image. Appendix A shows the formulation and reward.

Measuring the gain directly

That last result pointed to a new question: if the learned policy ignores the image, how much is there to gain from adapting to it at all? Answering it directly tells me whether the learning problem was worth solving: if an image's own best schedule is barely better than a shared one, no policy that chooses where to place steps can gain much by adapting to that image. Existing per-image scheduling methods are evaluated by aggregate FID (GITS) or through learned predictors, whose results also reflect prediction accuracy (INDIS, Yu et al.). Pairing each image's searched schedule with a shared one instead isolates the value of the schedule itself. Without training any policy, I fix the number of steps and compare each image's own best schedule with the single schedule optimized for the average across images. A schedule $M = \{t_0, \ldots, t_B\}$ places B steps between fixed start and end times, with step lengths $h_i = |t_{i+1} - t_i|$; both kinds of schedule always use the same B.

Evidence 1: exact optima of a surrogate cost

For $K = 256$ images per model, a surrogate cost sums each image's estimated local errors over the steps:

$$E_k(M) = \sum_{i=0}^{B-1} c_k(t_i)\, h_i^2.$$

Here $c_k(t)$, the image's *error profile*, measures how much local error its trajectory accumulates near time t. I estimate it from the curvature (second differences) of the image's 200-step DDIM trajectory; the square reflects DDIM being a first-order method. Normalising each profile to sum to one ($\tilde{c}_k$) gives every image equal weight and, by the scaling result below, leaves each image's optimum unchanged. The shared schedule then minimises the average cost, each image's schedule minimises its own, and g_k is the percentage by which the shared schedule exceeds the image's optimum:

$$M^\star_{\mathrm{sh}} \in \arg\min_M \frac{1}{K}\sum_{k=1}^{K} E_{\tilde{c}_k}(M), \qquad M^\star_k \in \arg\min_M E_{\tilde{c}_k}(M),$$

$$g_k = 100\left(\frac{E_{\tilde{c}_k}(M^\star_{\mathrm{sh}})}{E_{\tilde{c}_k}(M^\star_k)} - 1\right).$$

Dynamic programming finds both optima exactly for this surrogate (not for true image error) on a 188-point grid.

- **Only the shape of the error matters.** I proved that multiplying a profile by any $a > 0$ leaves its optimal schedules unchanged, since $E_{ac}(M) = a\,E_c(M)$. Only where along the trajectory an image's error concentrates can change its optimal schedule.
- **Small typical cost, heavy tail.** Across five pretrained 256×256 latent diffusion models (LSUN Church, LSUN Bedroom, CelebA-HQ, FFHQ, ImageNet) at $B = 4$ and 8, images often prefer different schedules: 256 church images have 165 distinct optima at $B = 8$. Yet the median g_k is at most 3.13% in every setting, while for a few images the shared schedule costs up to 94% more.

Evidence 2: full DDIM sampling

The surrogate ignores how errors propagate, so I also measured generated images directly. On LSUN Church and Bedroom at $N = 4$ and 8 denoising steps, $D_k(M)$ is the LPIPS distance between image k sampled with schedule M and a 200-step reference from the same initial noise. Coordinate descent finds a shared schedule $\hat{M}_{\mathrm{sh},N}$ and, starting from it, a schedule $\hat{M}_{k,N}$ for each image. I compare the mean per-image gain Δ_N with the gain from moving to a separately searched $(N{+}1)$-step shared schedule:

$$\Delta_N = \frac{1}{K}\sum_{k=1}^{K}\left[D_k(\hat{M}_{\mathrm{sh},N}) - D_k(\hat{M}_{k,N})\right], \qquad R_N = \frac{\Delta_N}{\bar{D}(\hat{M}_{\mathrm{sh},N}) - \bar{D}(\hat{M}_{\mathrm{sh},N+1})}.$$

$R_N < 1$ means the per-image gain was smaller than the gain from one extra step, and all four settings fall below one (Figure 1). If quality improves roughly linearly near N, R_N also reads as a fraction of a step: per-image scheduling recovered 0.25–0.71 of one extra step.

Because the searches are local, these are the gains coordinate descent found, not the largest possible. The 3.13% above is a surrogate quantity, while R_N is measured on generated images; both indicate a small typical benefit.

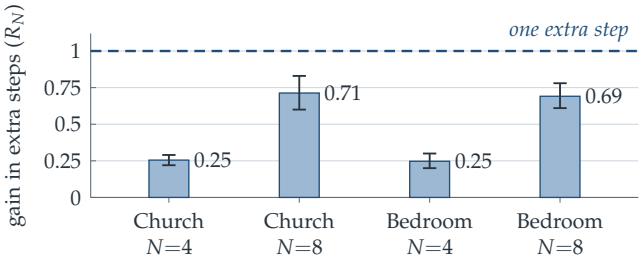


Figure 1. Per-image gain in extra denoising steps (R_N). Whiskers: 95% bootstrap intervals over images, with schedules and one-step gain held fixed. Dashed line: one extra step.

What I learned

At 4 and 8 steps, images often preferred different schedules, but in full sampling the average benefit of per-image optimization was smaller than the benefit of adding one step to the shared schedule. In these settings, schedule diversity alone did not justify per-image scheduling. Adaptive methods should therefore be compared against a shared schedule optimized for the same budget. One explanation I have not yet tested directly is that my learned samplers struggled because there was little per-image advantage to find. A small group of images benefits much more: for some, the shared schedule incurs up to 94% more surrogate cost than their own optimum. Adaptation could still pay off if those images can be identified cheaply, which is the next question I want to answer. I also want to test whether the advantage grows for text-conditioned or stochastic sampling, and, since step placement matters little for a typical image, whether adapting the compute inside each step, such as pruning or caching, gives per-image methods more room.

Code and reports

Repository: github.com/karthik6281/Adaptive-Step-Allocation-for-Diffusion-Sampling

- *How Much Is Per-Sample Step Placement Worth at a Fixed Budget?* (PDF): the full technical report.
- `step-placement/`: code for the step-placement study.
- `taed/`: code for TAED.
- `taed/team_5_report.pdf`: the original TAED Computer Vision course project report (team of four).

Appendix A. The reinforcement-learning formulation

Here j indexes the policy's denoise-or-skip decisions; in the main text, k indexes images.

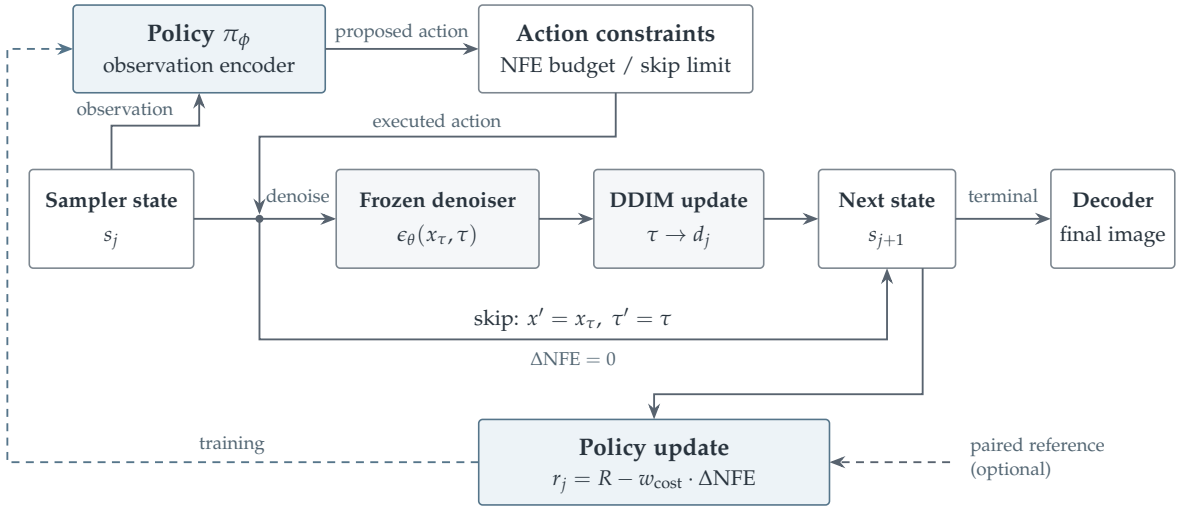


Figure A1. One sampling decision: the policy proposes denoise or skip, constraints (NFE budget, skip limit) set the executed action, and rewards update only the policy.

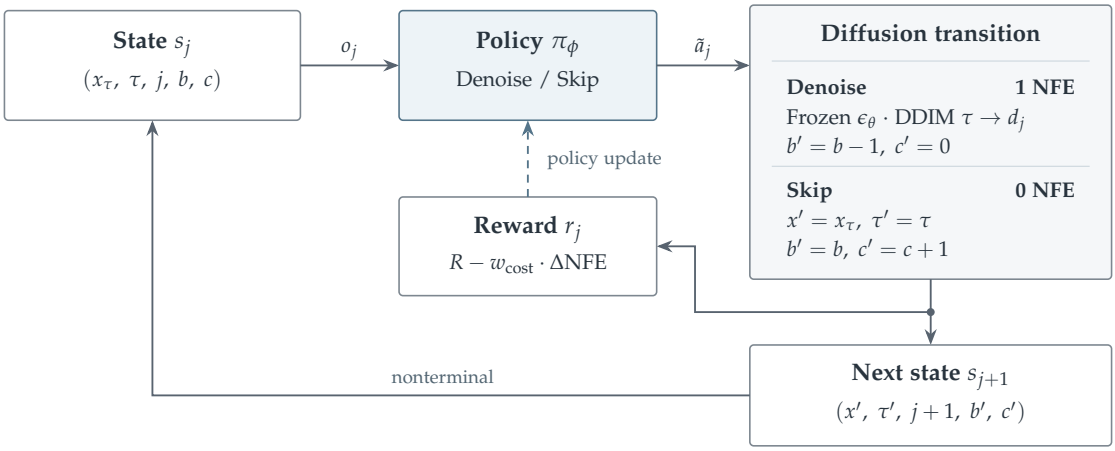


Figure A2. The MDP: the state holds the latent, current time τ, decision index j, remaining budget b and consecutive-skip count c; denoising costs one network evaluation (NFE) and skipping costs none.

The per-step reward combines the quality terms with a cost penalty:

$$r_j = R_j - w_{\text{cost}}\,\Delta\text{NFE}_j, \qquad R_j = \sum_i \lambda_i R_{i,j}.$$

R_j is a weighted sum of the quality terms, given only at the final step in the sparse variant and at every step in the dense one; ΔNFE_j is 1 when decision j denoises and 0 when it skips.